



Zadanie rekrutacyjne na stanowisko Full Stack Developer

Departament IT

Spis treści

Spis treści	2
1 Opis projektu	4
2 Założenia techniczne	5
2.1 Backend	5
2.2 Frontend	5
3 Role użytkowników	6
3.1 Użytkownik	6
3.2 Zarządca	6
3.3 Administrator (opcjonalnie)	6
4 Model biznesowy	7
4.1 Wniosek parkingowy	7
4.2 Statusy wniosku	7
5 Minimalny zakres projektu	8
5.1 Backend — wymagania obowiązkowe	8
5.1.1 Autoryzacja	8
5.1.2 Endpointy autoryzacji	8
5.1.3 Endpoint szlabanu	8
5.1.4 Endpointy nadzorcy	8
5.1.5 Struktura projektu backend	9
5.2 Frontend — wymagania obowiązkowe	9
5.2.1 Composables	9
5.2.2 Middleware Nuxt	9
5.2.3 Wymagania UI	9
6 Zakres optymalny	10
6.1 Backend	10
6.2 Frontend	10
6.3 DevOps / jakość kodu	10
7 Funkcjonalności opcjonalne	11
7.1 System wjazdu i wyjazdu	11
7.2 Panel administratora	11

7.3	Zaawansowana walidacja	11
8	Oczekiwania wobec rozwiązania.....	12
9	Przykładowy flow użytkownika	13

1 Opis projektu

Wspólnota mieszkaniowa „Euro Park” posiada wielopoziomowy parking dla mieszkańców. Ze względu na ograniczoną liczbę miejsc parkingowych administratorzy osiedla chcą zdigitalizować proces składania i obsługi wniosków o przydział miejsca parkingowego.

Celem projektu jest stworzenie aplikacji webowej składającej się z backendu oraz frontendu, która umożliwi:

- rejestrację i logowanie użytkowników,
- składanie wniosków o miejsce parkingowe,
- zarządzanie statusem wniosków przez nadzorców,
- weryfikację dostępu samochodu do szlabanu na podstawie numeru rejestracyjnego,
- opcjonalnie zarządzanie użytkownikami i monitorowanie wjazdów/wyjazdów.

Projekt powinien zostać wykonany w architekturze klient-serwer z wykorzystaniem REST API.

2 Założenia techniczne

2.1 Backend

Wymagane technologie:

- Python min 3.12+
- FastAPI
- SQLAlchemy
- Authentication
- Pydantic
- Alembic (mile widziane)
- PostgreSQL lub SQLite (lub inna relacyjna baza danych)

2.2 Frontend

Wymagane technologie:

- Nuxt 3 lub Nuxt 4
- Vue 3 Composition API
- TypeScript (mile widziany)
- fetch, useFetch
- Middleware Nuxt
- Composables

3 Role użytkowników

3.1 Użytkownik

Zwykły użytkownik systemu.

Może:

- zarejestrować konto,
- zalogować się,
- złożyć wniosek,
- sprawdzić status swoich wniosków,
- edytować wniosek, jeśli:
 - jest w statusie PENDING,
 - lub został odesłany do poprawy (NEEDS_CHANGES).

3.2 Zarządca

Nadzorca parkingu.

Może:

- przeglądać wszystkie wnioski,
- zaakceptować wniosek,
- odrzucić wniosek,
- odesłać do poprawy wraz z komentarzem,
- unieważniać dostęp (opcjonalnie).

3.3 Administrator (opcjonalnie)

Administrator systemu.

Może:

- dodawać użytkowników,
- usuwać użytkowników,
- zmieniać role użytkowników.

4 Model biznesowy

4.1 Wniosek parkingowy

Wniosek powinien zawierać:

- numer rejestracyjny pojazdu,
- preferowane piętro parkingu,
- status wniosku,
- datę utworzenia,
- komentarz nadzorcy (opcjonalnie),
- powiązanie z użytkownikiem.

4.2 Statusy wniosku

Przykładowe statusy:

- PENDING
- APPROVED
- REJECTED
- NEEDS_CHANGES

5 Minimalny zakres projektu

5.1 Backend — wymagania obowiązkowe

5.1.1 Autoryzacja

Implementacja:

- rejestracji,
- logowania,
- ochrony endpointów.

5.1.2 Endpointy autoryzacji

- Rejestracja
- Login

5.1.3 Endpoint szlabanu

POST /barrier/check-access

Dostęp powinien być przyznawany wyłącznie dla zaakceptowanych wniosków.

5.1.4 Endpointy nadzorcy

- Lista aplikacji,
- Akceptacja aplikacji,
- Odrzucenie aplikacji,
- Przekazanie aplikacji do poprawy (wymagany komentarz nadzorcy).

5.1.5 Struktura projektu backend

Przykładowa struktura:

backend/

```
├── app/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── models/
│   ├── schemas/
│   ├── middleware/
│   ├── core/
│   └── main.py
├── tests/
├── .env
├── requirements.txt
└── README.md
```

5.2 Frontend — wymagania obowiązkowe

5.2.1 Composables

useAuth() obsługuje logowania, rejestracji, wylogowanie, pobierania aktualnego użytkownika.

useApplications() obsługuje wszystkich operacji na wnioskach.

5.2.2 Middleware Nuxt

auth.ts

Ochrona tras wymagających logowania.

5.2.3 Wymagania UI

- czytelny interfejs,
- obsługa błędów (np. toast).

6 Zakres optymalny

Elementy oceniane dodatkowo:

6.1 Backend

- Alembic migrations,
- Docker / Docker Compose,
- testy jednostkowe,
- paginacja,
- filtrowanie,
- sortowanie,
- role-based access control,
- Swagger/OpenAPI/ReDocs customization,
- refresh token,
- rate limiting,
- logger.
- Walidacja danych wejściowych
 - walidacja danych wejściowych,
 - walidacja numeru rejestracyjnego (minimum regex),
 - obsługa błędów HTTP.

6.2 Frontend

- TypeScript,
- komponenty wielokrotnego użytku,
- skeleton loaders,
- toast notifications,
- SSR-friendly architecture,
- Middleware guest.ts - przekierowanie zalogowanego użytkownika z /login i /register.

6.3 DevOps / jakość kodu

- ESLint / Prettier,
- Ruff / Black,
- README z instrukcją uruchomienia.

7 Funkcjonalności opcjonalne

7.1 System wjazdu i wyjazdu

Endpoint:

- POST /barrier/entry
- POST /barrier/exit

Możliwe funkcjonalności:

- lista aktualnie zaparkowanych pojazdów,
- blokada podwójnego wjazdu,
- historia wjazdów i wyjazdów,
- limit czasu postoju.

7.2 Panel administratora

- zarządzania użytkownikami,
- nadawania ról,
- blokowania kont.

7.3 Zaawansowana walidacja

- pełna walidacja polskich numerów rejestracyjnych,
- blacklista numerów,
- ograniczenie liczby aktywnych wniosków.

8 Oczekiwania wobec rozwiązania

Oceniane będą:

- architektura projektu,
- jakość kodu,
- znajomość REST API,
- poprawne użycie metod HTTP,
- bezpieczeństwo,
- czytelność frontendu,
- walidacja,
- obsługa błędów,
- umiejętność pracy z rolami użytkowników,
- jakość README i dokumentacji.

9 Przykładowy flow użytkownika

1. Użytkownik rejestruje konto.
2. Loguje się.
3. Składa wniosek o miejsce parkingowe.
4. Wniosek trafia do kolejki oczekujących.
5. Nadzorca:
 - a. akceptuje,
 - b. odrzuca,
 - c. lub odsyła do poprawy.
6. Użytkownik widzi status.
7. Szlaban pyta API o dostęp